

MERN STACK

E-Commerce Web App

Roman Urdu Mein — Zero Se Hero Tak

CHAPTER 2

INTERMEDIATE

THEORY + CODE

ROMAN URDU

CHAPTER 2

Express.js aur HTTP — Pehla Web Server

HTTP kya hai? Server kaise banta hai? Routes aur Middleware ka raaz!

Is Chapter Mein Aap Seekhenge:

- HTTP kya hai aur kaise kaam karta hai
- Express.js install aur pehla server banana
- Routes — alag alag URLs handle karna
- GET, POST, PUT, DELETE methods samajhna
- JSON data bhejhna aur receive karna
- Middleware ka concept — ek bohat powerful idea
- CORS, Body Parser aur Error Handling
- Thunder Client se API test karna

■ Chapter 1 Recap

Node.js install, package.json, npm init, variables, data types — yeh sab aana chahiye. Agar nahi aaya toh pehle Chapter 1 dobara dekho!

TABLE OF CONTENTS

1.0 HTTP Kya Hai?

- 1.1 Client aur Server — do kirdar
- 1.2 HTTP Request kya hoti hai?
- 1.3 HTTP Response kya hoti hai?
- 1.4 Status Codes — 200, 404, 500 ka matlab

2.0 Express.js Kya Hai?

- 2.1 Express install karna
- 2.2 Pehla Express server
- 2.3 app.listen() — server start karna

3.0 HTTP Methods

- 3.1 GET — Data maangna
- 3.2 POST — Naya data bhejna
- 3.3 PUT — Data update karna
- 3.4 DELETE — Data mitana
- 3.5 Real life examples

4.0 Routes Banana

- 4.1 Route kya hoti hai?
- 4.2 Route parameters (:id)
- 4.3 Query parameters (?key=value)
- 4.4 ShopEase ke routes plan karna

5.0 JSON aur Request/Response

- 5.1 JSON kya hai?
- 5.2 res.json() — JSON bhejna
- 5.3 req.body — Data receive karna
- 5.4 express.json() middleware

6.0 Middleware Ka Concept

- 6.1 Middleware kya hai?
- 6.2 Built-in middleware
- 6.3 Custom middleware banana
- 6.4 CORS kya hai aur kyun zaroori hai

7.0 Error Handling

- 7.1 try-catch blocks
- 7.2 Custom error middleware
- 7.3 HTTP error status codes

8.0 Thunder Client se Testing

- 8.1 Thunder Client setup
- 8.2 GET request test karna
- 8.3 POST request test karna — JSON body ke saath

9.0 Summary & Practice

- 9.1 Chapter recap
- 9.2 Practice exercises
- 9.3 Chapter 3 preview

SECTION 1 — HTTP Kya Hai?

1.0 — HTTP Kya Hai?

Socho tum ek restaurant mein ho. Tum waiter se kehte ho: 'Mujhe biryani chahiye.' Waiter kitchen jaata hai aur biryani lekar wapas aata hai. Exactly aise hi internet kaam karta hai! **HTTP (HyperText Transfer Protocol)** woh rules hain jiske zariye computer ek doosre se baat karte hain.

Restaurant	Internet	Kya Hota Hai?
Tum (customer)	Browser (Chrome)	Jo maangta hai
Waiter	HTTP	Dono ke beech mein message le jaata hai
Kitchen	Server (Node+Express)	Request sun ke kaam karta hai
Biryani	Response (JSON/HTML)	Jo waapas aata hai
Menu	API Routes	Kya kya order ho sakta hai

1.1 — Client aur Server

Internet mein do kirdar hote hain: **Client** aur **Server**.

■ CLIENT	■ SERVER
Browser (Chrome, Firefox) ya Mobile App	Humara Node.js + Express server
Request karta hai — maangta hai	Request sun ke response deta hai
<i>'Mujhe products ki list do'</i>	<i>'Lo yeh rahi products ki list: [...]'</i>

1.2 — HTTP Request Kya Hoti Hai?

Jab browser server ko kuch maangta hai, woh ek **HTTP Request** bhejta hai. Is request mein yeh sab hota hai:

Part	Example	Kya Batata Hai?
Method	GET	Kya karna hai — maangna, bhejna, update, delete?

URL	/api/products	Kahan jaana hai — konsa resource chahiye?
Headers	Content-Type: application/json	Extra info — data format, auth token, etc.
Body	{ name: 'Shoes', price: 500 }	POST/PUT mein actual data jo bhejna hai

1.3 — HTTP Response Kya Hoti Hai?

Server request sun ke **HTTP Response** bhejta hai. Isme hota hai:

Part	Example	Matlab
Status Code	200	Kaam hua ya nahi — 200 = OK, 404 = Not Found
Headers	Content-Type: application/json	Response ka format
Body	{ success: true, data: [...] }	Actual data jo wapas aa raha hai

1.4 — HTTP Status Codes — Zaroor Yaad Karo!

Status code ek 3-digit number hai jo batata hai ke request successful rahi ya nahi. Jaise school mein marks hote hain — 100 perfect, 0 fail!

Code	Naam	Matlab / Kab Aata Hai?
200	OK	Sab theek — request successful
201	Created	Naya data successfully bana gaya (POST ke baad)
400	Bad Request	Request mein kuch galat tha — data incomplete ya wrong format
401	Unauthorized	Login nahi kiya — pehle login karo
403	Forbidden	Login kiya hai lekin permission nahi — admin ka page hai
404	Not Found	Jo maanga woh mila nahi — route ya data exist nahi karta
422	Unprocessable	Data format sahi hai lekin values wrong hain
500	Internal Server Error	Server mein koi bug hai — humari galti!

■ TIP

2xx = Success (green light) | 4xx = Client ki galti | 5xx = Server ki galti. Yeh codes hamesha yaad rakhna — debugging mein bohat kaam aate hain!

SECTION 2 — Express.js Kya Hai?

2.0 — Express.js Kya Hai?

Node.js mein hum server bana sakte hain — lekin bahut zyada code likhna padta hai. **Express.js** ek **framework** hai jo Node.js ke upar bana hai aur server banana super asaan kar deta hai. Jaise ek ready-made skeleton — hum sirf apna kaam add karte hain!

Bina Express ke vs Express ke saath comparison:

	Bina Express (Pure Node)	Express ke Saath
Code Lines	50+ lines sirf ek route ke liye	5-10 lines — bas!
Routing	Manually if/else likhna padta	<code>app.get()</code> , <code>app.post()</code> — clean!
Middleware	Khud banana padta	Built-in aur npm se available
Error Handling	Bahut complicated	Simple aur organized

2.1 — Express Install Karna

Terminal mein shopease folder mein yeh command chalao:

```
# Express install karo
npm install express

# Package.json check karo — dependencies mein express aa gaya hoga:
# "dependencies": {
#   "express": "^4.18.2"
# }
```

2.2 — Pehla Express Server Banana!

server.js file mein yeh code likho (pehle wala sab delete karo):

```
// Step 1: Express import karo
const express = require('express');

// Step 2: Express application banana
const app = express();
```

```
// Step 3: Port define karo — server kahan sunegaa
const PORT = 5000;

// Step 4: Pehli route banana — '/' matlab home page
app.get('/', (req, res) => {
  res.send('ShopEase Server Chal Raha Hai! ■');
});

// Step 5: Server start karo
app.listen(PORT, () => {
  console.log(`Server port ${PORT} pe chal raha hai`);
  console.log(`Link: http://localhost:${PORT}`);
});
```

2.3 — Har Line Ka Matlab (Bohat Important!)

<code>require('express')</code>	require() se hum koi bhi installed package import karte hain. 'express' woh package hai jo humne install kiya.
<code>const app = express()</code>	Express function call karne se ek 'application object' milta hai. Yeh app hi hamara poora server hai.
<code>const PORT = 5000</code>	PORT number woh darwaza hai jahan server sunegaa. 5000 common hai backend ke liye. (3000 React ke liye)
<code>app.get('/', ...)</code>	GET method se '/' (home) route banana. Jab browser http://localhost:5000 kholega, yeh chalega.
<code>(req, res) => { }</code>	Arrow function hai. req = request (jo browser ne bheja), res = response (jo hum bhejenge).
<code>res.send(...)</code>	Browser ko kuch bhejna — text, HTML, ya JSON. Yeh response complete karta hai.
<code>app.listen(PORT, ...)</code>	Server ko PORT par start karo. Jo callback function hai woh server start hone ke baad chalega.

2.4 — Server Chalao!

Terminal mein:

npm run dev

Output aayega:

Server port 5000 pe chal raha hai

Link: <http://localhost:5000>

Ab browser mein jao aur likho:

<http://localhost:5000>

Screen pe dikhega: ShopEase Server Chal Raha Hai!

■ SAHI KIYA

MUBARAK HO! Tumhara pehla web server chal raha hai! localhost matlab tumhara apna computer. :5000 matlab port number — darwaza. Browser ne request bheji, Express ne response diya!

SECTION 3 — HTTP Methods (GET, POST, PUT, DELETE)

3.0 — HTTP Methods Kya Hain?

Methods batate hain ke hum kya karna chahte hain. Jaise orders mein hum kehte hain 'lao', 'rakh do', 'badal do', 'hata do' — aise hi HTTP mein 4 main methods hain. Inhe **CRUD** kehte hain:

Method	CRUD	Kaam	Example
GET	Read	Data maangna / padhna	Products ki list lao
POST	Create	Naya data banana / bhejna	Naya product add karo
PUT	Update	Purana data update karna	Product ki price badal do
DELETE	Delete	Data delete karna	Product hata do

3.1 — GET Route — Data Maangna

GET route tab use hota hai jab hum kuch maangna chahte hain — data read karna. Koi data nahi bhejte, sirf request karte hain.

HTTP Method: GET

```
// GET — Saare products lao

app.get('/api/products', (req, res) => {

  // Abhi ke liye dummy data — baad mein database se aayega

  const products = [

    { id: 1, name: 'Nike Shoes', price: 5000, category: 'Footwear' },

    { id: 2, name: 'Levi Jeans', price: 3500, category: 'Clothing' },

    { id: 3, name: 'Leather Bag', price: 8000, category: 'Bags' },

  ];

  res.status(200).json({

    success: true,

    count: products.length,
```

```
    data: products
  });
});
```

■ TIP

Browser se directly test kar sako — GET request easy hai!

3.2 — POST Route — Naya Data Bhejna

POST route tab use hota hai jab hum kuch naya banana chahte hain. Hum body mein data bhejte hain — jaise ek form fill karke submit karna.

HTTP Method: POST

```
// POST – Naya product add karo

app.post('/api/products', (req, res) => {
  // req.body mein bheja hua data hoga
  const { name, price, category } = req.body;

  // Validation – zaroor check karo
  if (!name || !price) {
    return res.status(400).json({
      success: false,
      message: 'Name aur price dono zaroori hain!'
    });
  }

  // Naya product object banao
  const newProduct = { id: Date.now(), name, price, category };

  res.status(201).json({
    success: true,
    message: 'Product successfully add ho gaya!',
    data: newProduct
  });
});
```

■ TIP

POST request browser se directly test nahi ho sakti — Thunder Client use karo!

3.3 — PUT Route — Data Update Karna

PUT route tab use hota hai jab hum koi existing cheez update karna chahte hain. URL mein ID dete hain (konsa update karna hai) aur body mein naya data.

HTTP Method: PUT

```
// PUT — Product update karo (ID se pehchano)
app.put('/api/products/:id', (req, res) => {
  const { id } = req.params;    // URL se ID lo
  const { name, price } = req.body; // Body se naya data lo

  // Yahan database mein update hoga (baad mein seekhenge)
  res.status(200).json({
    success: true,
    message: `Product ${id} update ho gaya!`,
    data: { id, name, price }
  });
});
```

■ TIP

:id ek route parameter hai — yeh dynamic value hai. /api/products/5 mein 5 id hai.

3.4 — DELETE Route — Data Mitana

DELETE route kisi cheez ko mitane ke liye use hota hai. URL mein ID dete hain — bas, body ki zaroorat nahi.

HTTP Method: DELETE

```
// DELETE — Product mitao
app.delete('/api/products/:id', (req, res) => {
  const { id } = req.params;

  // Yahan database se delete hoga
  res.status(200).json({
    success: true,
```

```
    message: `Product ${id} delete ho gaya!`  
  });  
});
```

■ YAAD RAHO

DELETE careful se use karo — data permanently chala jaata hai!

SECTION 4 — Routes Banana

4.0 — Route Kya Hoti Hai?

Route ek **address** hai — jaise ghar ka address hota hai, waise hi server pe har kaam ka ek address hota hai. Har address pe alag kaam hota hai.

ShopEase ke liye humari routes ka plan:

HTTP Method	Route URL	Kya Karta Hai?
GET	/api/products	Saare products ki list
GET	/api/products/:id	Ek specific product (ID se)
POST	/api/products	Naya product add karo
PUT	/api/products/:id	Product update karo
DELETE	/api/products/:id	Product delete karo
POST	/api/users/register	User register karo
POST	/api/users/login	User login karo
GET	/api/users/profile	User ka profile dekho
GET	/api/orders	Orders ki list
POST	/api/orders	Naya order place karo

4.1 — Route Parameters (:id)

Route parameter ek **dynamic value** hai URL mein — colon (:) se start hota hai. Jaise /api/products/42 mein 42 product ID hai.

```
// Route parameter — :id dynamically change ho sakta hai
app.get('/api/products/:id', (req, res) => {
  const productId = req.params.id; // URL se ID extract karo
  // Agar URL hai: /api/products/42
  // Toh productId = '42' hoga

  console.log('Product ID maanga gaya:', productId);
});
```

```
res.json({
  success: true,
  message: `Product ${productId} ki detail`,
  data: { id: productId, name: 'Nike Shoes', price: 5000 }
});
```

4.2 — Query Parameters (?key=value)

Query parameters URL ke aakhir mein ? ke baad aate hain. Filtering, searching, sorting ke liye use hote hain.

```
// URL example: /api/products?category=shoes&sort=price
app.get('/api/products', (req, res) => {
  const { category, sort, page = 1, limit = 10 } = req.query;
  // category = 'shoes'
  // sort = 'price'
  // page = 1 (default)
  // limit = 10 (default)

  console.log('Filter:', category, 'Sort by:', sort);

  res.json({
    success: true,
    filters: { category, sort },
    page: Number(page),
    limit: Number(limit)
  });
});
```

■ TIP

req.params → Route parameters (:id) | req.query → Query parameters (?key=val) |
req.body → Request body (POST/PUT mein). Yeh teeno bohat important hain!

SECTION 5 — JSON aur Request/Response

5.0 — JSON Kya Hai?

JSON (JavaScript Object Notation) ek format hai data share karne ka. Jaise hum Urdu mein baat karte hain — computers JSON mein baat karte hain! Poori internet JSON use karti hai data exchange ke liye.

JSON ka example:

```
{
  "name": "Nike Air Max",
  "price": 15000,
  "inStock": true,
  "category": "Footwear",
  "images": ["img1.jpg", "img2.jpg"],
  "specs": {
    "color": "Black",
    "size": [7, 8, 9, 10, 11]
  }
}
```

Rule	Sahi	Galat
Keys mein double quotes	"name": "Ali"	name: 'Ali' (single quotes nahi)
Strings mein double quotes	"city": "Karachi"	"city": 'Karachi'
Numbers bina quotes	"price": 500	"price": "500"
Booleans lowercase	"active": true	"active": True
Last item mein comma nahi	{ "a": 1 }	{ "a": 1, }

5.1 — express.json() Middleware — Body Parse Karna

Jab POST/PUT request mein JSON data aata hai, Express ise automatically nahi samajhta. **express.json()** middleware ko add karna hoga — yeh JSON parse kar ke req.body mein daal deta hai.


```

const express = require('express');

const app = express();

// ZAROOR ADD KARO — Body parser middleware
// Bina is ke req.body undefined hoga!

app.use(express.json());

// Ab POST route mein req.body available hoga
app.post('/api/products', (req, res) => {
  console.log(req.body); // { name: 'Shoes', price: 5000 }
  res.json({ success: true, received: req.body });
});

```

■ YAAD RAHO

`app.use(express.json())` hamesha routes se PEHLE likhna — warna kaam nahi karega. Order bohat important hai Express mein!

5.2 — Structured Response Pattern

Consistent response structure banana professional habit hai. ShopEase mein hum yeh pattern follow karenge:

```

// SUCCESS response pattern
res.status(200).json({
  success: true,
  message: 'Products successfully mile!', // optional
  count: data.length,                  // optional
  data: data                           // actual data
});

// ERROR response pattern
res.status(400).json({
  success: false,
  message: 'Galat request — name aur price zaroori hain',
  error: error.message // debugging ke liye
});

```

SECTION 6 — Middleware Ka Concept

6.0 — Middleware Kya Hai?

Middleware ek **function** hai jo **request** aur **response** ke beech mein chalta hai. Socho airport security ki tarah — pehle security check hota hai, phir andar jaate ho. Har middleware Request dekhta hai, kuch kaam karta hai, phir agle step ko bhejta hai.

Middleware ka flow kuch aisa hota hai:



6.1 — Custom Middleware Banana

Middleware sirf ek function hota hai jo 3 arguments leta hai: req, res, next.

```
// Custom Logger Middleware — har request log karo
const logger = (req, res, next) => {
  const time = new Date().toISOString();
  console.log(`[${time}] ${req.method} ${req.url}`);
  // next() call karna zaroori hai!
  // Bina next() ke request yahan ruk jaayegi
}
```

```

    next();
  };

  // Middleware globally apply karo (saari routes pe)
  app.use(logger);

  // Ya sirf ek route pe apply karo:
  app.get('/api/products', logger, (req, res) => {
    res.json({ data: 'products here' });
  });

```

■ YAAD RAHO

`next()` bhoolna sabse common mistake hai! Agar `next()` call nahi kiya toh request hang ho jaayegi — browser wait karta rahega, response kabhi nahi aayega.

6.2 — CORS Kya Hai aur Kyun Zaroori Hai?

CORS (Cross-Origin Resource Sharing) browser ki ek security feature hai. Jab React (`localhost:3000`) Express (`localhost:5000`) se data maangta hai — browser block kar deta hai: 'alag origin hai, nahi deta!' CORS enable karne se yeh problem solve ho jaati hai.

```
# cors package install karo
```

```
npm install cors
```

```

const cors = require('cors');

// CORS middleware – routes se pehle
app.use(cors({
  origin: 'http://localhost:3000', // React ka address
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  credentials: true // Cookies allow karo
}));

// Ya development mein sab allow karo (production mein nahi!)
app.use(cors());

```

SECTION 7 — Error Handling

7.0 — Error Handling Kyun Zaroori Hai?

Errors hamesha aate hain — database disconnect ho gaya, user ne galat data diya, file nahi mili. Agar hum errors handle nahi karein, poora server crash ho jaata hai! Professional apps mein **graceful error handling** hoti hai.

7.1 — try-catch Block

try-catch ek safety net hai. try block mein code chalao — agar kuch galat hua toh catch block mein pakad lo.

```
app.get('/api/products/:id', async (req, res) => {  
  try {  
    // Yahan kuch bhi fail ho sakta hai  
    const { id } = req.params;  
  
    // Check karo ID valid hai ya nahi  
    if (!id || isNaN(id)) {  
      return res.status(400).json({  
        success: false,  
        message: 'Valid product ID dena zaroori hai!'  
      });  
    }  
  
    // Successful response  
    res.status(200).json({ success: true, data: { id, name: 'Shoes' } });  
  }  
  
  } catch (error) {  
    // Koi bhi unexpected error yahan aa jaayega  
    console.error('Error aaya:', error.message);  
    res.status(500).json({  
      success: false,  
      message: 'Server mein koi masla hai, baad mein try karo'  
    });  
  }  
})
```

```
});
```

7.2 — Global Error Handler Middleware

Ek centralized error handler banao jo sab errors ko handle kare. Yeh special middleware 4 parameters leta hai (err, req, res, next):

```
// Global error handler — SARI routes ke BAAD likhna
app.use((err, req, res, next) => {
  console.error('Global Error:', err.stack);

  const statusCode = err.statusCode || 500;
  const message = err.message || 'Kuch galat ho gaya!';

  res.status(statusCode).json({
    success: false,
    message: message,
    // Production mein stack trace mat dikhao!
    stack: process.env.NODE_ENV === 'development' ? err.stack : undefined
  });
});

// 404 handler — koi bhi route match nahi hua
app.use('*', (req, res) => {
  res.status(404).json({
    success: false,
    message: `Route ${req.originalUrl} exist nahi karta`
  });
});
```

■ YAAD RAHO

Global error handler hamesha SARI routes ke baad add karo — warna kaam nahi karega. Express mein order bohat important hai!

SECTION 8 — Thunder Client se API Testing

8.0 — Thunder Client Kya Hai?

GET routes hum browser se test kar sakte hain — lekin POST, PUT, DELETE ke liye body mein data bhejna padta hai. Browser se yeh mumkin nahi. **Thunder Client** VS Code extension hai jo Postman ki tarah kaam karta hai — directly VS Code ke andar!

8.1 — Thunder Client Setup

1. VS Code mein Extensions (Ctrl+Shift+X) kholna
2. 'Thunder Client' search karo aur install karo
3. Left sidebar mein thunder icon aa jaayega — click karo
4. 'New Request' button pe click karo

8.2 — GET Request Test Karna

Step	Kya Karna Hai?
1	Method dropdown mein GET select karo
2	URL mein likho: <code>http://localhost:5000/api/products</code>
3	Send button dabao
4	Response mein JSON data dikhega — status 200 hoga

8.3 — POST Request Test Karna (JSON Body ke Saath)

Step	Kya Karna Hai?
1	Method mein POST select karo
2	URL: <code>http://localhost:5000/api/products</code>
3	Body tab pe click karo
4	'JSON' option select karo
5	JSON body likho: <code>{ "name": "Nike Shoes", "price": 5000, "category": "Footwear" }</code>
6	Send dabao — response mein naya product dikhega

8.4 — Poora server.js — Abhi Tak Ka Code

Ab tak jo seekha sab ek file mein:

[illegible]

```

});

// POST new product
app.post('/api/products', (req, res) => {
  const { name, price, category } = req.body;

  if (!name || !price) return res.status(400).json({ success: false, message: 'Name aur price zaroori!' });

  const newProduct = { id: Date.now(), name, price, category };
  products.push(newProduct);

  res.status(201).json({ success: true, data: newProduct });
});

// PUT update product
app.put('/api/products/:id', (req, res) => {
  const idx = products.findIndex(p => p.id === Number(req.params.id));
  if (idx === -1) return res.status(404).json({ success: false, message: 'Product nahi mila' });

  products[idx] = { ...products[idx], ...req.body };
  res.status(200).json({ success: true, data: products[idx] });
});

// DELETE product
app.delete('/api/products/:id', (req, res) => {
  products = products.filter(p => p.id !== Number(req.params.id));
  res.status(200).json({ success: true, message: 'Product delete ho gaya!' });
});

// 404 handler
app.use('*', (req, res) => {
  res.status(404).json({ success: false, message: 'Route nahi mila!' });
});

// Global error handler
app.use((err, req, res, next) => {
  res.status(500).json({ success: false, message: err.message });
});

```


[illegible]

SECTION 9 — Summary, Practice & Next Steps

9.0 — Is Chapter Ka Recap

■ HTTP	HyperText Transfer Protocol — computers ke baat karne ke rules
■ Status Codes	200=OK, 201=Created, 400=Bad Req, 401=Unauth, 404=NF, 500=Server Error
■ Express.js	Node.js framework — server banana super asaan
■ app.get/post/put/delete	HTTP methods — CRUD operations ke liye routes
■ req.params	:id se URL ke dynamic values nikalna
■ req.query	?key=value se query parameters nikalna
■ req.body	POST/PUT request mein bheja hua JSON data
■ express.json()	Body parser — req.body available karta hai
■ Middleware	Request-Response ke beech chalna wala function — next() zaroori
■ CORS	Cross-origin requests allow karna — React + Express ke liye
■ Error Handling	try-catch + global error middleware — professional approach
■ Thunder Client	VS Code mein API testing — POST/PUT/DELETE ke liye

9.1 — Practice Exercises

■ Easy Level

1. server.js banao — Express install karo, pehla server chalao
2. GET route banao: /api/hello — 'Hello ShopEase!' return karo
3. Browser mein <http://localhost:5000/api/hello> open karo
4. Status codes yaad karo — ek card pe likho!

■ Medium Level

1. 5 dummy products ki array banao
2. GET /api/products — saari list return karo
3. GET /api/products/:id — ek specific product
4. Logger middleware add karo
5. Thunder Client se dono routes test karo

■ Challenge Level

1. POST /api/products — validation ke saath (naam/price required)
2. PUT /api/products/:id — existing product update karo
3. DELETE /api/products/:id — product hatao
4. 404 aur global error handler add karo
5. CORS middleware add karo
6. Sab Thunder Client se thoroughly test karo

9.2 — Chapter 3 Ka Preview

MongoDB aur Mongoose — Real Database Connect Karo!

- MongoDB kya hai — NoSQL database ka concept
- MongoDB Atlas — free cloud database setup
- Mongoose kya hai — ODM (Object Document Mapper)
- Schema aur Model banana
- CRUD operations with real database
- Validation aur default values
- Server.js ko real database se connect karna

Shabash! Chapter 2 Complete Ho Gaya! ■

Ab tumhara Express server chal raha hai, routes ban gaye, middleware samajh aa gaya, aur Thunder Client se testing bhi kar li. Yeh sab real backend development hai — wahi kaam jo professional developers karte hain! **Chapter 3** mein hum real database connect karenge — shayad sabse exciting chapter!